SROS2 DoS 实验答辩小抄

1. 一句话解释这个实验

这个实验不是为了绕过 SROS2 权限，而是验证：

```
SROS2 能阻止未授权发布 command topic，
但它不等于完整的可用性防御。
```

攻击机不创建合法 `/cmd_vel_in` publisher，而是向 DDS/RTPS 使用的 UDP 端口发送高频 RTPS-like 包，观察 secure controller 到 Gazebo 的控制链路是否出现延迟、中断或资源压力。

2. 为什么打 7400-7423 这些端口

ROS2 默认通信底层使用 DDS。当前项目使用的是：

```
RMW_IMPLEMENTATION=rmw_fastrtps_cpp
FASTDDS_BUILTIN_TRANSPORTS=UDPv4
ROS_DOMAIN_ID=0
```

DDS/RTPS 会使用 UDP 端口做 participant discovery 和 user data 通信。常见端口范围从 `7400` 开始，后续端口会随着 domain、participant index、内置 discovery endpoint、用户数据 endpoint 增加。

本实验不是手写固定端口，而是让攻击脚本在运行时扫描本机 `/proc/net/udp*`，找出当前打开的 DDS 相关端口：

```
7400 <= port <= 7600
```

这次实际发现并攻击的是：

```
7400, 7410, 7411, 7412, 7413, 7414, 7415,
7416, 7417, 7418, 7419, 7420, 7421, 7422, 7423
```

答辩时可以这样说：

```
这些端口不是随便选的，而是 ROS2/FastDDS 当前运行时实际打开的 DDS/RTPS UDP socket。
脚本用发现机制确认端口后再攻击，避免只攻击一个固定端口导致误判。
```

3. 这些端口大概在做什么

可以用概念解释，不需要背精确公式：

端口类型	作用
discovery 端口	DDS participant 发现、节点互相知道对方存在
built-in endpoint 端口	DDS 内部元数据交换，例如 reader/writer 发现
user data 端口	ROS2 topic 数据最终通过 DDS writer/reader 传输

在本项目里，相关数据路径是：

```
Docker controller
  publishes /cmd_vel_in
        |
        v
WSL host cmd_vel_relay
  subscribes /cmd_vel_in
  publishes /cmd_vel
        |
        v
ros_gz_bridge -> Gazebo TurtleBot3
```

所以如果 DDS/RTPS 层被高频 UDP traffic 干扰，可能表现为：

- discovery 或 endpoint 匹配变慢
- topic 数据短时间收不到
- controller 发布线程被调度影响
- relay 一段时间收不到 `/cmd_vel_in`
- robot 出现停顿或控制延迟

4. 这次实验的关键证据

证据目录：

```
logs/experiments/log_driven_validation/20260416_043331/sros2_dos/
```

4.1 攻击流量确实发生

`dos_attack.log`：

```
Target ports: [7400, 7410, ..., 7423]
Duration: 25.0s
Threads per port: 3
Finished. Sent 613326 packets in 26.2s
avg 23386 pkt/s
```

解释：

```
攻击机容器在 25 秒内发送了约 61.3 万个 RTPS-like UDP 包，
平均约 2.34 万包每秒。
```

4.2 攻击机资源占用明显升高

`resource_monitor.log`：

```
tb3-attacker 257.70%
tb3-attacker 252.04%
tb3-attacker 263.05%
```

解释：

```
attacker 容器 CPU 超过 250%，说明攻击脚本确实产生了高负载发包行为。
```

4.3 controller 出现 3 秒级发布卡顿

`controller_session.log`：

```
1776285263.296  published=220
1776285266.325  published=222
```

正常 controller 是 10 Hz，3 秒应该发布约 30 条。这里 3 秒只增加 2 条。

第二个卡顿：

```
1776285298.296  published=542
1776285301.402  published=545
```

解释：

```
攻击没有让 controller 永久失效，但产生了短时 control-loop stall。
```

4.4 relay 检测到 `/cmd_vel_in` 中断

`test.log`：

```
No /cmd_vel_in for 2.93s; published zero /cmd_vel
No /cmd_vel_in for 2.91s; published zero /cmd_vel
```

解释：

```
relay 一段时间没有收到 controller 的输入，所以 watchdog 主动发送 zero /cmd_vel。
这说明 DoS 产生了短时 command-path interruption。
```

5. 最终结论应该怎么说

推荐说法：

```
本实验没有证明攻击者可以绕过 SROS2 权限，也没有造成机器人永久失效。
但是在 SROS2 secure baseline 下，DDS/RTPS UDP flood 造成了约 3 秒级的控制输入中断。
因此，SROS2 能解决认证和授权问题，但不能单独解决可用性问题。
```

更短版本：

```
SROS2 blocks unauthorized command injection, but availability still needs separate
protection.
```

6. 老师可能问的问题

Q1: 这是不是 DDoS?

答：

```
严格说这不是分布式 DDoS，而是单攻击机 DoS / UDP flood。
我的实验里 attacker 是一个 Docker 容器，所以应该叫 DoS 更准确。
如果有多个 attacker 容器或多台机器同时发包，才更接近 DDoS。
```

Q2: 为什么打 7400-7423?

答：

```
ROS2 使用 DDS/RTPS 作为底层通信协议。FastDDS 默认使用 UDP 端口，常见范围从 7400 开始。
实验不是硬编码只打 7400，而是在运行时扫描 /proc/net/udp*，找出当前 DDS 使用的 7400-
7600 范围端口。
这次运行实际发现的是 7400 和 7410-7423，所以攻击这些端口。
```

Q3: 这些端口是不是只负责 discovery？如果只打 discovery，为什么会影响控制？

答：

```
DDS 端口既可能包含 discovery，也可能包含 user data 或 built-in endpoint traffic。
我的实验不是精确区分每个端口的语义，而是对当前 DDS runtime 打开的一组端口做
availability pressure test。
观测结果显示 controller 和 relay 出现短时中断，所以至少对控制路径产生了可观测影响。
```

Q4: SROS2 不是加密认证了吗，为什么还能被 DoS？

答：

```
SROS2 解决的是身份认证、权限控制、消息完整性和保密性。
但是任何安全协议都仍然要接收网络包、解析头部、调度线程或丢弃无效包。
这些处理本身会消耗资源，所以 SROS2 不能天然防止 availability attack。
```

Q5: 攻击有没有绕过 SROS2？

答：

```
没有。这个实验不声称绕过 SROS2。
SROS2 仍然成功阻止了 attacker 创建合法 /cmd_vel_in publisher。
DoS 实验说明的是另一个问题：即使攻击者不能发合法 command，也可能通过 DDS/RTPS 层流量压力
造成短时可用性退化。
```

Q6: 机器人真的受影响了吗？

答：

```
日志里能看到两类影响：
1. controller 在 10Hz 发布时出现约 3 秒卡顿；
2. relay 检测到 2.9 秒没有收到 /cmd_vel_in，并发布 zero /cmd_vel。

所以它不是控制劫持，也不是永久停机，而是短时控制输入中断。
```

Q7: 为什么轨迹 CSV 没有生成？

答：

```
第一次自动化采集时，外部 trajectory recorder 没有以 /gazebo SROS2 enclave 启动。
在 SROS2 enforce 模式下，没有正确 enclave 的普通 ros2 node 看不到安全域里的 /odom。
所以 CSV 没有数据。
```

```
但是 Gazebo 进程内部的 trajectory marker node 是在 /gazebo enclave 下运行的，它成功收
到 /odom 并生成 marker。
下一步改进是让 recorder 也使用 /gazebo enclave。
```

Q8: 为什么说是 partial availability impact，而不是 attack success?

答:

```
因为攻击没有完全打挂系统，也没有持续阻断控制。
严谨地说，它造成的是短时 command-path interruption。
所以我把结论写成 partial availability impact，避免夸大攻击效果。
```

Q9: 有什么防御方案?

答:

```
SROS2 负责认证和授权，但 availability 需要额外防御:
1. 网络隔离，只允许可信主机访问 DDS 端口;
2. 防火墙限制 7400-7600/udp 的访问范围;
3. 容器 CPU / 网络速率限制;
4. DDS 配置限制 discovery 范围;
5. watchdog，像本项目 relay 一样，输入中断时立即发 zero command;
6. 监控 DDS socket、topic rate、controller publish rate。
```

Q10: 这个实验和 MITM 有什么区别?

答:

```
MITM 是攻击者进入通信路径，目标是观察、延迟、丢弃或篡改合法通信。
DoS 是攻击者不一定在路径中，而是通过大量流量造成资源压力或通信中断。
SROS2 可以保护消息内容不被篡改，但不能自动保证网络路径和中间件资源一定可用。
```

7. 答辩时推荐的讲法

可以按下面顺序讲:

1. 先讲 SROS2 防御成功：attacker 不能创建 `/cmd_vel_in` publisher。
2. 再转折：但是安全不只有权限，还有 availability。
3. 解释 DDS/RTPS 端口：ROS2 底层通过 FastDDS UDP 端口通信，实验扫描并攻击实际打开的 `7400-7423`。
4. 展示攻击量：`613,326 packets / 26.2s / 23,386 pkt/s`。
5. 展示影响：controller 10Hz 出现 3 秒卡顿，relay 检测 `/cmd_vel_in` 中断。
6. 下结论：不是绕过 SROS2，而是说明 SROS2 需要配合网络隔离、rate limit 和 watchdog。

8. 不要这样说

避免这些表述：

```
SROS2 被攻破了。
攻击者绕过了 SROS2。
DoS 完全控制了机器人。
这是 DDoS 攻击。
```

更准确的表述：

```
SROS2 access control remained effective.
The UDP flood caused short availability degradation in the secure control path.
```